

Relazione Tecnica:

Progetto “Fast Food” — Programmazione Web e Mobile 2025/2026

1. Analisi Tecnica Dettagliata per file:

1.0 middlewares/authenticateUser.js:

Funzione: `authenticateUser(req, res, next)`

Scopo:

- Validare JWT da header Authorization e popolare `req.user`.

Flusso:

1. Controllo presenza/formato Bearer.
2. Estrazione token.
3. `jwt.verify(token, JWT_SECRET)`.
4. Validazione `payload.userId` e `payload.role`.
5. Assegnazione `req.user`.
6. `next()` o 401 in errore.

1.1 middlewares/authorizeRistoratore.js:

Funzione: `authorizeRistoratore(req, res, next)`

Scopo:

- Consentire accesso solo a ruolo ristoratore.

Flusso:

1. Verifica `req.user.role`.
2. Se diverso da `ristoratore` ritorna 403.
3. Altrimenti `next()`.

1.2 node_modules/jsonwebtoken/verify.js:

Funzione esportata: `module.exports = function (jwtString, secretOrPublicKey, options, callback)`

Scopo:

- Verificare struttura, firma e claim del JWT.

Flusso:

1. Normalizza opzioni.
2. Valida formato JWT (3 parti).
3. Decodifica token.
4. Recupera/valida chiave e algoritmo.

5. Verifica firma crittografica.
6. Verifica claim temporali (nbf, exp) e semantici (aud, iss, sub).

1.3 public/assets/auth.js:

Ruolo:

Modulo Frontend auto-eseguito (IIFE) per gestione sessione client, validazione token lato browser e protezione pagine per ruolo.

Struttura generale:

- Tutto il file è incapsulato in una IIFE: `(function () { ... })()`;
- Evita di mischiarsi con il global scope, esponendo solo API minime su window.

Funzioni dettagliate:

1. getCurrentPath()

Scopo: Ottenere `window.location.pathname` con fallback `/`.

2. normalizePathname(pathname)

Scopo: Rimuovere query/hash dal path per confronti coerenti.

3. isAuthPage(pathname)

Scopo: Verificare se il path è una pagina di autenticazione (`/login.html`, `/register.html`, ecc.).

4. saveLastVisitedPath(pathname)

Scopo: Salvare in `localStorage` l'ultima pagina non-auth visitata.

Gestione errori: try/catch per gestire browser policy/storage non disponibile.

5. restoreLastVisitedPath(payload)

Scopo: Dopo login, ripristinare la pagina precedente o usare fallback per ruolo (`/ristoratore/home.html` o `/cliente/home.html`).

6. decodeJwtPayload(token)

Scopo: Decodificare payload JWT lato client senza verifica firma.

Passi: Split token, conversione `base64url` → `base64`, padding, `atob`, `JSON.parse`.

Nota: Utile per UX/navigation state; non sostituisce verifica server-side.

7. showAuthIssue(message)

Scopo: Uniformare logging warning auth e ritorno false.

8. inferRoleFromPath(pathname)

Scopo: Inserire ruolo (ristoratore/cliente) dal prefisso URL.

9. `getRoleScopedValue(baseKey, role)`

Scopo: Leggere da `localStorage` una chiave specifica per ruolo (`token_rirestoratore`, ecc.).

10. `setRoleScopedValue(baseKey, role, value)`

Scopo: Scrivere in `localStorage` valori namespace per ruolo.

11. `validateSession(expectedRole)`

Scopo: Funzione centrale di validazione sessione client.

Flusso:

1. Determina ruolo attivo (atteso o inserito dal path).
2. Recupera token role-scoped o fallback globale.
3. Decodifica payload e verifica campi minimi (`role`, `userId`).
4. Se si verifica il "mismatch" del ruolo, si tenta il fallback con token globale.
5. Sincronizza storage globale e per ruolo (`token`, `role`, `userId`).
6. Salva percorso corrente.
7. Ritorna il payload valido o null.

12. `bootstrapNavigationState()` (IIFE interna)

Scopo: All'avvio della pagina, valida sessione e prova il ripristino ultima pagina.

API esposte su window:

`window.handleAuthIssue(reason):`

- Wrapper di `showAuthIssue`.

`window.ensureAuthenticated():`

- Ritorna `true` se la sessione è valida, altrimenti `false` + warning.

`window.ensureAuthenticatedRole(expectedRole):`

- Valida autenticazione + coerenza del ruolo rispetto alla pagina.
- Previene redirect involontari su refresh in caso di "mismatch".

1.4 `public/assets/modern-theme.css:`

Ruolo:

Foglio riguardante lo stile principale del tema UI moderno (tipografia, layout, componenti, colori).

1.5 `public/assets/responsive.css:`

Ruolo:

Regole responsive/adattive per viewport differenti.

1.6 public/assets/logo.png e public/assets/Favicon.png:

Ruolo:

Asset grafici branding/interfaccia.

1.7 public/cliente/*.html:

Ruolo generale:

Le pagine public/cliente implementano i flussi frontend del cliente (home, menù, offerte, carrello, ordini), riutilizzando ../assets/auth.js per i controlli di autenticazione/ruolo.

Analisi funzione-per-funzione:

public/cliente/home.html:

- **fetchConControlloToken(url, options = {}):** wrapper fetch con header Authorization e controllo errori token.
- **ricercaRistoranti(queryName, queryAddress):** invoca endpoint ricerca ristoranti e applica filtri input utente.
- **mostraRistoranti(lista):** rendering DOM elenco ristoranti e stati vuoto/errore.

public/cliente/menù.html:

- **fetchConControlloToken(url, options = {}):** fetch autenticato con gestione sessione invalida.
- **caricaMenù():** carica dati menù dal Backend e prepara dataset UI.
- **mostraPiatti(lista):** rendering schede piatto.
- **addToCart(piatto):** chiamata API per aggiunta al carrello e feedback utente.
- **generaCategorie(piatti):** costruisce dinamicamente categorie filtro dal dataset.
- **applicaFiltri():** filtra lista piatti secondo categoria/ricerca/altri criteri UI.

public/cliente/offerte.html:

- **getIdAsString(idValue):** normalizzazione ID (ObjectId/string) per confronti coerenti.
- **readArrayFromLocalStorage(key):** lettura robusta array da localStorage con fallback.
- **getMealIdAsString(meal):** estrazione ID piatto normalizzato.
- **fetchWithAuth(url, options):** wrapper fetch autenticato.
- **normalizeMeal(meal):** normalizzazione shape dati pasto per rendering/filtri.
- **isMealAllowedByPreference(meal):** verifica compatibilità offerta con preferenze utente.
- **buildScenarioContent(hasOffers, hasPreferences, hasPreferenceMatches):** produce contenuto UI in base allo scenario.
- **addToCart(meal):** aggiunge offerta al carrello.
- **renderGroupedOffers(meals, emptyMessage):** rendering offerte raggruppate per ristorante/scenario.
- **loadUserPreferences():** recupero preferenze utente.
- **loadOffers():** recupero offerte e avvio pipeline rendering.

public/cliente/carrello.html:

- **fetchConControlloToken(url, options = {}):** fetch autenticato con fallback su sessione non valida.
- **recuperaCarrello():** recupera stato carrello corrente.
- **recuperaIndirizzoUtente():** legge indirizzo utente necessario per consegna.
- **mostraMessaggio(tipo, testo):** mostra feedback UI (errore/successo/info).

- *estraiComponentiIndirizzo(indirizzo)*: parsing indirizzo testuale in componenti utili.
- *corrispondeValore(valoreUtente, possibiliValori)*: helper matching semantico valori indirizzo.
- *validaIndirizzoReale(indirizzo)*: validazione indirizzo via servizio esterno/Nominatim.
- *mostraCarrello()*: rendering completo carrello, raggruppamento per ristorante e totali.
- *rimuoviDalCarrello(mealId)*: rimozione item dal carrello.
- *handler click orderBtn*: validazioni checkout, costruzione payload ordini e invio API.
- *(async function init() { ... })()*: bootstrap iniziale pagina (load dati + rendering).

public/cliente/ordini.html:

- *fetchConControlloToken(url, options = {})*: wrapper fetch autenticato.
- *formatEuro(value)*: formattazione prezzo in valuta EUR.
- *normalizzaClasseStato(stato)*: mapping stato ordine → classe CSS.
- *creaDettagliPasti(order)*: genera markup dettagli pasti per ordine.
- *creaCardOrdine(order)*: genera card HTML ordine completa.
- *parseDataOrdine(dataOrdine)*: parsing robusto data ordine per sorting.
- *caricaOrdiniCliente()*: fetch storico ordini, ordinamento e rendering.
- *confermaConsegna(orderId, buttonElement)*: conferma ricezione ordine via API e aggiorna UI.

1.8 public/ristoratore/*.html:

Ruolo generale:

Le pagine *public/ristoratore* implementano i flussi operativi del ristorante: creazione/gestione ristorante, gestione menù e bacheca/offerte, CRUD → (Create, Read, Update, Delete - Creazione, Lettura, Aggiornamento, Eliminazione) per i piatti personalizzati, monitoraggio ordini e statistiche.

Analisi funzione-per-funzione:

public/ristoratore/home.html:

- *fetchConControlloToken(...)*: wrapper fetch autenticato.
- *caricaDatiRistorante()*: recupera dati del ristorante del ristorante loggato.
- *caricaOrdini(restaurantId)*: recupera e renderizza riepilogo ordini recenti.

public/ristoratore/creaRistorante.html:

- *mostraFeedback(tipo, messaggio)*: feedback UX form.
- *resetCampo(...), evidenziaCampoErrore(...), resetValidazione()*: utilità validazione UI.
- *trovaCampoDaErrore(messaggio)*: mapping errore backend → campo form.
- *validaForm()*: validazione completa client-side prima del submit.
- *fetchConControlloToken(...)*: invio autenticato payload creazione.

public/ristoratore/gestioneRistorante.html:

- *fetchConControlloToken(...)*: wrapper auth.
- *caricaDatiRistorante()*: load dati correnti e rendering form.
- *confrontaValori(a,b)*: helper confronto robusto.
- *nessunaModificaRistorante(payload, originale)*: evita update inutili se nessuna modifica.

public/ristoratore/gestioneMenù.html:

- Funzioni storage/selezione: *getIdAsString*, *readArrayFromLocalStorage*, *getBachecaStorageKey*, *getBachecaIds*, *salvaBachecaRistorante*.
- Funzioni dati/render: *fetchConControlloToken*, *caricaRistorante*, *caricaMenu*, *renderizzaMenu*, *creaCardPiatto*, *mostraDettagliPiatto*.
- Funzioni di Elaborazione in blocco: *toggleMealSelection*, *ripristinaPiattiSelezionati*, *selezionaTuttiPiatti*, *updateDeleteButtonLabel*, *eliminaPiattiSelezionati*, *aggiungiPiattiInBacheca*.

public/ristoratore/gestioneBacheca.html:

- Include quasi la stessa base di gestioneMenù + logica offerte.
- Funzioni offerte: *aggiornaOfferteMeals*, *aggiornaOffertaMeal*, *aggiungiPiattiAlleOfferte*, *rimuoviPiattiDalleOfferte*.
- Funzione di eliminazione: *eliminaPiattiSelezionati*.
- Helper messaggi: *getNomePiatto*, *getMessaggioPiattiGialInOfferta*, *getMessaggioPiattiAggiunti*.

public/ristoratore/piattiGenerici.html:

- *caricaPiattiGenerici()*, *caricaCategorie()*: dataset catalogo generico.
- *displayGenericMealsPage(page)*, *updatePaginationControls()*: paginazione lato client.
- *bindMealSelectionEvents*, *toggleMealSelection*, *updateAddSelectedButtonLabel*: selezione multipla.
- *aggiungiPiattoAlMenu(idMeal)*, *aggiungiPiattiSelezionatiAlMenu()*: inserimento nel menù del ristorante.
- *applicaFiltri()*: filtri categoria/ricerca.

public/ristoratore/piattoPersonalizzato.html:

- *fetchConControlloToken(...)*, *caricaRistorante()*: contesto auth/ristorante.
- **Handler submit form** (*addEventListener('submit', ...)*): costruzione payload piatti personalizzati e POST backend.

public/ristoratore/modificaPiattoPersonalizzato.html:

- *renderCards()*: render elenco/edit action piatti personalizzati.
- *popolaForm(piatto)*: bind valori selezionati nel form modifica.
- *caricaDettagliPiatto(idMeal)*: fetch dettaglio singolo piatto.
- *caricaRistorante()*: contesto owner e filtri.

public/ristoratore/ordini.html:

- Formattazione/render: *estraiTimestampOrdine*, *formatEuro*, *renderPrezzoOrdine*, *renderSubtotaleOrdine*, *normalizzaClasseStato*, *creaDettagliPasti*, *creaCardOrdine*.
- Dati e azioni: *fetchConControlloToken*, *caricaOrdiniRistoratore*, *cambiaStatoOrdine*, *confermaLetturaNotificheConsegna*.
- Gestione separata ordini attivi/storico e transizioni degli stati.

public/ristoratore/statistiche.html:

- *fetchConControlloToken(...)*: accesso endpoint statistiche.
- *renderListItem(label, value, suffix)*: rendering righe (KPI – Key Performance Indicators).
- *formatDateLabel(dateString)*: formattazione etichette temporali.
- *caricaStatistiche()*: organizzazione fetch (KPI – Key Performance Indicators) e rendering dashboard.

1.9 public/* .html:

Ruolo:

Le pagine nel root di public coprono accesso, onboarding, uscita sessione, profilo utente e landing page applicativa.

Analisi funzione-per-funzione:

public/login.html:

- **Handler submit** (`loginForm.addEventListener("submit", ...)`): valida input base, invia credenziali al backend, salva token/ruolo in storage e gestisce redirect in base al ruolo.

public/register.html:

- **updateFields()**: abilita/disabilita campi in base al ruolo scelto.
- **renderPreferenze(categorie)**: rendering chip preferenze alimentari.
- **validaUsername(username)**: validazione formato username.
- **getPreferenzeSelezionate()**: estrazione preferenze attive dalla UI.
- **resetErrors(), showFieldError(...), clearFieldError(...), focusAndScrollToField(...)**: gestione UX errori form.
- **validateForm(values)**: validazione completa campi registrazione.
- **caricaCategoriePreferenze()**: carica categorie da API (Meals) per popolare preferenze.
- **Handler eventi click/input/blur/invalid/submit**: organizzano validazione progressiva + invio registrazione.

public/logout.html:

- **Script inline di logout**: rimozione token/dati sessione da storage e redirect a pagina di ingresso/login.

public/profilo.html:

- **goBack()**: ritorno contestuale alla home di ruolo.
- **fetchConControlloToken(...)**: wrapper fetch autenticato.
- **mostraMessaggio(...), pulisciMessaggio(...), mostraEsitoAggiornamentoProfilo(...)**: gestione feedback UI.
- **Validator**: **validaEmail, validaUsername, validaTelefono, validaPiva, validaPassword**.
- **Preferenze**: **renderPreferenze(...), getPreferenzeSelezionate(), caricaCategoriePreferenze()**.
- **trovaDifferenze(payload)**: calcola modifiche reali prima dell'update.
- **caricaDatiProfilo()**: fetch dati utente e popolamento form.
- **Handler submit/click**:
 - Aggiornamento profilo,
 - Cambio password,
 - Cancellazione account.

1.10 routes/*.js:

Ruolo:

La cartella routes contiene la logica API REST principale (controller Express), con middleware di autenticazione/autorizzazione e accesso a MongoDB tramite `req.app.locals.db`.

routes/users.js:

Funzioni:

- *sanitizePreferenze(preferenze)*: normalizza ed elimina i duplicati delle preferenze alimentari.
- *sanitizeMetodoPagamento(metodoPagamento)*: normalizza metodo di pagamento.

Endpoint principali:

- *GET /users*: elenco utenti (projection senza password).
- *POST /users/register*: registrazione con validazioni per ruolo e dati obbligatori.
- *POST /users/login*: autenticazione + emissione JWT.
- *GET /users/me*: profilo autenticato.
- *PUT /users/me*: aggiornamento profilo con validazioni e controllo coerenza.
- *PUT /users/me/password*: cambio password autenticato.
- *DELETE /users/me*: cancellazione account e cleanup/refresh correlato.

routes/meals.js:

Funzione:

- *parseCsv(value)*: parser CSV robusto per query param (ingredienti/allergeni).

Endpoint principali:

- *GET /meals*: listing con filtri avanzati (categoria, prezzo, ingredienti, allergeni, area, offerte).
- *PUT /meals/offerte*: aggiornamento massivo flag/sconto offerte (ristoratore).
- *GET /meals/:id*: dettaglio pasto.
- *POST /meals*: creazione piatto (owner ristorante).
- *PUT /meals/:id*: modifica piatto con controlli owner.
- *DELETE /meals/:id*: cancellazione piatto autorizzata.

routes/restaurants.js:

Funzioni:

- *parseOrderDateToDate(rawValue)*: parsing robusto per la data dell'ordine.
- *getYyyyMmDd(dateValue)*: normalizzazione della data YYYY-MM-DD.

Endpoint principali:

- *GET /restaurants/search*: ricerca per nome/indirizzo.
- *GET /restaurants/by-meal*: lookup ristoranti a partire da meals.
- *GET /restaurants*: elenco ristoranti.
- *GET /restaurants/statistics*: (KPI – Key Performance Indicators) ristorante (ordini, fatturato, top pasti, timeline).
- *GET /restaurants/:restaurantId*: dettaglio ristorante + menù popolato.

- **POST /restaurants**: creazione del ristorante autenticata/registrata.
- **PUT /restaurants/:restaurantId**: aggiornamento ristorante “owner-only”.
- **DELETE /restaurants/:restaurantId**: eliminazione ristorante “owner-only”.

routes/orders.js:

Funzioni:

- **geocodeAddress(address)**: geocoding indirizzo tramite provider esterno.
- **haversineKm(coord1, coord2)**: fallback distanza geodetica (lunghezza del segmento che in linea d’aria calcola la distanza tra due punti).
- **getDrivingRouteMetrics(coordFrom, coordTo)**: routing stradale (km/min) via OSRM.
- **calculateOrderPreparationMinutes(order)**: stima tempi preparazione backlog.
- **updateOrderStatus(req,res)**: logica condivisa per la transizione dello stato dell’ordine.

Endpoint principali:

- **POST /orders**: creazione ordini da carrello (anche multi-ristorante).
- **GET /orders**: storico ordini (cliente/ristoratore in base al ruolo).
- **PUT /orders/notifiche-consegna/ack**: pacchetto ACK (riscontro/conferma) notifiche consegna lato ristoratore.
- **GET /orders/:id**: dettaglio ordine con controlli accesso.
- **PUT /orders/:id/consegna**: conferma consegna lato cliente.
- **PUT /orders/:id/stato** (tramite handler dedicato): avanzamento stato operativo.

routes/carts.js:

Endpoint principali:

- **GET /carts/me**: recupero carrello utente.
- **POST /carts/me/items**: aggiunta item al carrello con merge quantità.
- **DELETE /carts/me/items/:mealId**: rimozione item.
- **DELETE /carts/me**: svuotamento carrello.
- **PUT /carts/add**: incremento quantità item.
- **PUT /carts/remove**: decremento/rimozione quantità item.

2. utils/*.js:

Ruolo:

I file in utils contengono logica trasversale riusabile: configurazione applicativa fail-fast e validazione indirizzi con **OpenStreetMap/Nominatim**.

utils/config.js:

Funzione: extractDbNameFromMongoUri(uri)

Scopo:

- Estrarre il nome DB dalla URI Mongo quando MONGODB_DB non è esplicitato.

Comportamento:

- Se URI assente ritorna null.
- Tenta parse con new URL(uri).
- Estrae pathname senza / iniziale e prende il primo segmento come nome DB.
- In caso di URI invalida ritorna null (catch silenzioso).

Altra logica rilevante del modulo:

- dotenv.config(): carica variabili .env.
- Costruisce requiredEnvVars e missingEnvVars.
- Se mancano variabili critiche (**MONGODB_URI**, **MONGODB_DB**, **JWT_SECRET**, **JWT_EXPIRES_IN**) lancia errore bloccante.
- Esporta oggetto config tipizzato per bootstrap server (**PORT**, **URI DB**, **segreti JWT**).

utils/addressValidation.js:

Funzione: normalizeValue(value = "")

Scopo: Normalizzare stringhe (trim/lowercase/rimozione accenti/spazi multipli) per confronti robusti.

Funzione: extractAddressParts(rawAddress = "")

Scopo: Fare parsing semantico dell'indirizzo utente.

Output: address, chunks, cap, streetChunk, cityChunk, hasStreetNumber.

Funzione: addressMatches(parsedInput, nominatimAddress = {})

Scopo: Verificare coerenza fra indirizzo utente parsato e addressdetails restituiti da Nominatim.

Dettagli:

- Normalizza tipi strada (via/viale/piazza/...).
- Confronta strada, città e CAP con matching tollerante.
- Ritorna true solo se tutti i campi chiave corrispondono.

Funzione export: validateAddressWithOpenStreetMap(rawAddress, options = {})

Scopo:

- Validare un indirizzo italiano (**default expectedCountryCode = "it"**) con controlli sintattici + verifica remota OSM.

Flusso:

1. Parsing input con extractAddressParts.

2. Validazioni preliminari (lunghezza minima, civico, CAP, città).
3. Costruzione URL Nominatim (*jsonv2, addressdetails, limit, countrycodes*).
4. Chiamata fetch con timeout (**AbortController**).
5. Controllo response + parsing JSON risultati.
6. Ricerca di un risultato coerente tramite *addressMatches*.
7. Ritorno esito strutturato: { **valid: true, normalizedAddress, coordinates** } oppure { **valid: false, reason** }.
8. Cleanup timeout in finally.

Criticità operative:

- Dipende da disponibilità rete/provider esterno.
- Richiede gestione rate-limit e policy User-Agent lato produzione.

2.0 Root repository (Progetto-Fast-food-Programmazione-web-e-mobile-2025):

Ruolo generale:

La /root del progetto contiene endpoint Backend, metadata di dipendenze/build, documentazione generale e file di configurazione Git.

index.js (endpoint server):

Funzione: *normalizeMealDocument(meal)*

Scopo:

- Normalizzare i record seed "Meals" convertendo *_id* in *ObjectId* valido quando presente in formato export (*\$oid*).

Comportamento:

- Copia il documento ({ ...meal }).
- Se *_id.\$oid* è valido crea *new ObjectId(...)*.
- Altrimenti **rimuove** *_id* per lasciare che Mongo lo generi.

Funzione: *bootstrapInitialMeals(db)*

Scopo:

- Eseguire setup iniziale dello stesso valore della collection *meals* da *documents/meals.json*.

Comportamento:

1. Legge count documenti in *meals*.
2. Se non è vuota, esce subito.
3. Carica JSON seed.
4. Normalizza con *normalizeMealDocument*.
5. Inserisce batch con *insertMany(..., { ordered: false })*.

Funzione: startServer()

Scopo:

- Avvio completo applicazione Express + connessione MongoDB + bootstrap route/middleware.

Flusso:

1. Connessione Mongo tramite MongoClient usando **config.MONGODB_URI**.
2. Bind DB in **app.locals.db** + **ping** di verifica.
3. Esecuzione setup iniziale "Meals".
4. Registrazione route (**/users**, **/meals**, **/restaurants**, **/orders**, **/carts**).
5. Setup error middleware globale.
6. Avvio **app.listen(PORT)**.
7. In caso di errore bootstrap: **log** + **process.exit(1)**.

Altri file root:

- **package.json**: definisce script npm, dipendenze runtime/dev e metadata progetto.
- **package-lock.json**: lockfile per le versioni di dipendenze per Elementi in blocchi riproducibili.
- **.gitignore**: esclusione file/cartelle non versionabili (es. **node_modules**, **env locali**).
- **.gitattributes**: attributi Git per normalizzazione/testi/binari.

2.1 Flusso integrato complessivo:

1. **documents/swagger.js** produce **documents/swagger.json** tramite **swagger-autogen**.
2. Backend protegge le route via **authenticateUser** + **authorizeRistoratore**.
3. Frontend (**public/assets/auth.js**) gestisce stato sessione client e controllo accesso pagina lato UX.
4. **jsonwebtoken/verify.js** realizza la verifica token effettiva usata dal middleware server.
5. **meals.json** e asset statici (**css**, **png**, **pdf**) completano bootstrap e presentazione.